

Hindsight Experience Replay for Sparse-Reward Robotic Manipulation: Implementation, Reward Engineering, and Robustness Analysis

Dev Rawal

Abstract

This report investigates the application of Hindsight Experience Replay (HER) [1] to the `FetchPush` robotic manipulation task, where a 7-DOF Fetch robot arm must push a cube to a target position under a sparse binary reward. Without HER, Soft Actor-Critic (SAC) achieves only 6% success rate, confirming that sparse rewards are intractable for standard off-policy reinforcement learning. After implementing HER with the *future* goal-relabeling strategy ($k=4$), SAC+HER reaches **96%** success at 250K timesteps. SAC and Deep Deterministic Policy Gradient (DDPG) are further compared, both with HER; DDPG fails entirely (5% success), attributable to its deterministic exploration strategy and susceptibility to value overestimation. Two custom reward functions, dense distance-energy and approach-progress, are designed and evaluated; both fail to improve over sparse+HER, likely due to incompatibility between non-goal-recomputable reward components and HER’s relabeling procedure. Finally, policy robustness is evaluated under physics perturbations and domain randomization (DR); the nominal SAC+HER policy generalises well (80–100% across all shifts) and outperforms the DR-trained policy in most conditions. The key take-away is that HER alone is sufficient to make `FetchPush` tractable, and that reward engineering must be carefully designed to be HER-compatible.

1 Introduction & Background

Goal-conditioned RL and the sparse-reward problem. Reinforcement learning for robot control requires agents to map high-dimensional observations to continuous motor commands through trial and error. Many manipulation tasks are *goal-conditioned*: the agent must reach a target state that changes each episode. The natural reward for such tasks is sparse, returning 0 on success and -1 otherwise, creating a severe exploration problem. With random actions, the agent almost never receives a positive reward, leaving the Q-function with no useful gradient signal. In `FetchPush`, random policies achieve $\approx 5\%$ success due to fortuitous initial configurations, making standard SAC/DDPG unable to learn [4].

Hindsight Experience Replay. HER [1] resolves this by reframing failed episodes as successes toward the goal that *was* achieved. After each episode, HER retroactively substitutes the desired goal with states visited during the episode, recomputes rewards accordingly, and stores these virtual transitions alongside the originals. This guarantees that every episode produces at least k positive-reward transitions, converting the reward landscape from nearly flat to informative. HER requires off-policy algorithms (SAC, DDPG) because it modifies transitions stored in a replay buffer; on-policy methods cannot use it.

Scope of investigation. This report investigates four aspects of HER-based robot learning: (1) the mechanics and impact of HER on sparse-reward exploration in `FetchPush`; (2) the design and evaluation of custom reward functions intended to accelerate HER-based training; (3) a comparison of SAC and DDPG when combined with HER; and (4) the effect of introducing domain randomization during training on policy robustness to physics perturbations. These directions have direct practical relevance to deploying RL policies on real hardware, where reward design, algorithm choice, and sim-to-real transfer are key engineering decisions.

2 Method: HER Implementation

2.1 Data Structures

The `HERReplayBuffer` stores complete episodes rather than individual transitions. Each episode is represented as a list of tuples $(o_t, a_t, o_{t+1}, g_t^a, g^d)$, where g_t^a is the achieved goal and g^d is the desired goal. Episodes are appended to a fixed-size circular array; when the buffer reaches capacity, the oldest episode is evicted (FIFO overflow). This episode-level organisation is essential: the *future* strategy requires access to subsequent timesteps within the *same* episode, which would not be recoverable if transitions were stored individually.

2.2 The “Future” Strategy

For each transition at timestep t in an episode of length T , k future timesteps $t' \sim \mathcal{U}\{t, \dots, T\}$ are sampled. The achieved goal at each sampled t' , $g_{t'}^a$, is used as a virtual

desired goal for the transition at t . The reward is then recomputed as:

$$r' = \text{compute_reward_static}(g_t^a, g_t^d, \delta) \quad (1)$$

where $\delta=0.05$ m is the goal-achievement threshold. This strategy works because the agent *always* achieves some state; using that state as a goal guarantees at least one success per episode. With $k=4$, approximately $k/(k+1) = 80\%$ of sampled transitions are HER-relabelled, creating a dense stream of positive-reward experience.

2.3 Relabelling in `sample()`

When drawing a minibatch, each transition is independently relabelled with probability $k/(k+1)$. For relabelled transitions, the desired goal is replaced with the sampled future achieved goal and the reward is recomputed via Eq. (1). Original (non-relabelled) transitions, sampled with probability $1/(k+1)$, retain the true desired goal and original reward. The output is a `ReplayBufferSamples` namedtuple, ensuring the training loop requires no modification.

2.4 Key Design Decisions

Edge-case handling. For the final transition of an episode, there are strictly no "future" states available. Instead of excluding these transitions from the sampling pool, the sampling function allows the current timestep to act as its own future. Consequently, if a final transition is selected for HER relabeling, its desired goal is simply overwritten by its own achieved goal, resulting in a valid, positive-reward transition representing a terminal success state

Static reward decoupling. The reward function is implemented as a static method `compute_reward_static` with no access to environment state. HER relabels goals *after* the episode ends, when environment state is no longer available. Any reward component that depends on actions, velocities, or other state variables cannot be correctly applied to relabelled transitions.

Buffer management. The buffer tracks both a per-episode write pointer and a total episode count. Sampling is performed uniformly over all stored transitions across all stored episodes, ensuring that older episodes remain accessible until evicted.

3 Experimental Setup

3.1 Environment

The experiments use `FetchPushFlat`, a custom wrapper over `FetchPush` from Gymnasium-Robotics [6]. Observations are a 31-dimensional flat vector: 25 dimensions of robot state (joint positions, velocities, gripper state, object state), 3 dimensions for the desired goal (target x, y, z), and 3 for the achieved goal (object x, y, z). The action space is 4-dimensional continuous (3D end-effector velocity + gripper command, clipped to $[-1, 1]$). Each episode lasts 50 timesteps; the task is considered solved if the object is within 5 cm of the target.

3.2 Algorithm Configurations

Both SAC and DDPG are trained for 250K environment steps with HER ($k=4$), using a replay buffer capacity of 1M transitions. All runs are tracked via Weights & Biases. The DR policy is trained with physics parameters randomised at each episode reset (mass $\in [0.5, 2.0] \times$, friction $\in [0.5, 2.0] \times$, size $\in [0.8, 1.2] \times$).

3.3 Hardware

All experiments were conducted on a personal laptop equipped with an NVIDIA GeForce RTX 3060 (6 GB VRAM) GPU and an Intel Core i7 CPU. Training runs utilised the GPU where available.

3.4 Reward Functions

Sparse (baseline).

$$r_{\text{sparse}} = \begin{cases} 0 & \|g^a - g^d\|_2 < 0.05 \\ -1 & \text{otherwise} \end{cases} \quad (2)$$

No gradient toward the goal between timesteps; relies entirely on HER for learning signal.

Dense distance + energy (dense energy).

$$r_{\text{dense}} = -\|g^a - g^d\|_2 - \lambda_e \|a_{1:3}\|_2^2 \quad (3)$$

The first term provides a continuous gradient toward the goal at every timestep. The energy penalty ($\lambda_e=0.1$) discourages large actuations by penalising the squared L2 norm of the translational action dimensions (first 3 of 4), motivated by physical actuator limits on real hardware. Expected to improve sample efficiency over sparse+HER.

Approach + progress (approach.progress).

$$r_{\text{prog}} = -\|p_{\text{ee}} - p_{\text{obj}}\|_2 - \|g^a - g^d\|_2 + \beta \Delta d_t \quad (4)$$

where p_{ee} is the end-effector position, p_{obj} is the object position, and Δd_t is the reduction in object-goal distance since the previous step ($\beta=10.0$). The approach and push distance terms are weighted equally (1:1 ratio). This two-phase structure is intended to first guide the gripper to make contact, then reward directed pushing toward the goal.

4 Results

The following subsections report results across all four experimental conditions.

4.1 HER Impact

Table 1 shows the stark effect of HER on SAC performance. Without HER, SAC achieves 6% success, barely above random, despite 250K training steps. With HER, success jumps to 96%.

Table 1: Success rate before and after HER (SAC, sparse reward, 250K steps).

Algorithm	HER	Success %	Mean Return
SAC	No	6%	-47.00
SAC	Yes	96%	-15.64

4.2 Reward Function Comparison

Table 2 compares all reward variants using SAC+HER. Counterintuitively, both custom rewards fail to learn, achieving near-random performance. The sparse reward with HER remains the best-performing variant.

Table 2: SAC+HER performance for each reward variant (250K steps).

Reward	Success %	Return	Energy
Sparse	96%	-15.64	1.20
Dense+Energy	6%	-47.00	0.03
Approach+Progress	7%	-46.59	0.86

4.3 SAC+HER vs. DDPG+HER

Table 3 compares the two algorithms using the sparse reward (which proved best in Table 2). DDPG+HER fails entirely, matching the no-HER baseline, while SAC+HER achieves 96%. DDPG’s energy usage (3.00) is more than

twice that of SAC (1.20), indicating uncoordinated high-magnitude actuation rather than directed pushing.

Table 3: SAC+HER vs. DDPG+HER (sparse reward, 250K steps).

Algorithm	Success %	Return	Energy
SAC + HER	96%	-15.64	1.20
DDPG + HER	5%	-47.42	3.00

4.4 Robustness Under Domain Shifts

Table 4 reports success rates under physics perturbations for both the nominal SAC+HER policy and the DR-trained policy. The nominal policy is generally robust; low mass ($\times 0.5$) is the most damaging perturbation (80%). Notably, the DR-trained policy does not outperform the nominal policy and underperforms on several conditions.

Table 4: Robustness under domain shifts.

Shift	Multiplier	Nominal	DR
None	$\times 1.0$	96%	—
Friction	$\times 0.5$	97%	90%
	$\times 1.5$	100%	97%
	$\times 2.0$	98%	86%
Mass	$\times 0.5$	80%	68%
	$\times 1.5$	99%	92%
	$\times 2.0$	100%	92%
Size	$\times 0.8$	98%	94%
	$\times 1.2$	96%	90%

4.5 Training Curves

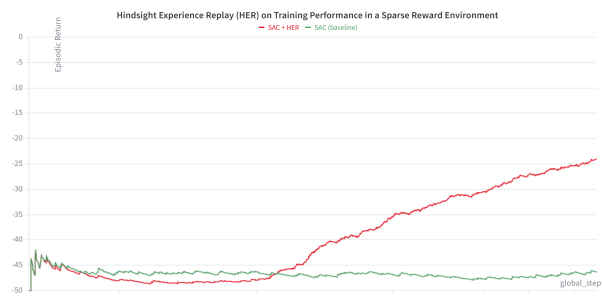


Figure 1: Training curves: episodic success rate vs. environment steps for SAC without HER and SAC+HER (sparse reward). HER begins improving performance around 110–120K steps.

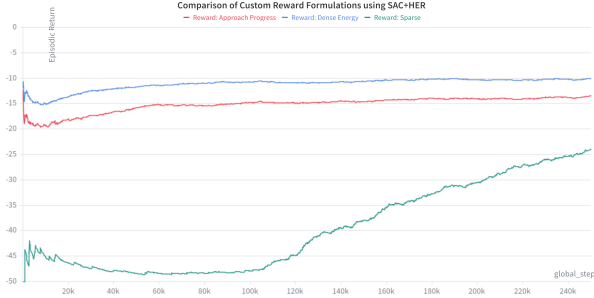


Figure 2: Training curves for all reward variants with SAC+HER.

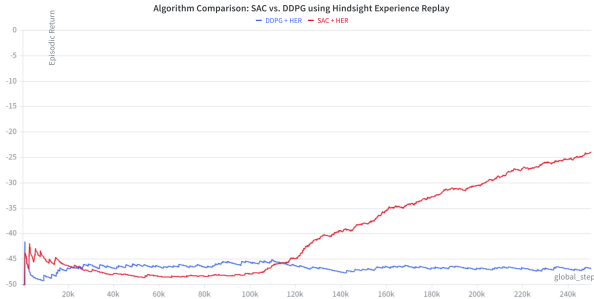


Figure 3: SAC+HER vs. DDPG+HER (sparse reward). DDPG fails to improve beyond the random baseline.

5 Analysis & Discussion

5.1 Why HER Transforms the Problem

Without HER, all 250K training steps produce near-uniform -1 reward, giving the Q-network no gradient to distinguish useful from useless actions. HER solves this by treating every visited state as a valid goal. With the *future* strategy and $k=4$, roughly 80% of sampled mini-batch transitions carry a positive reward, creating the dense learning signal necessary for Q-value differentiation.

The choice of k reflects a trade-off. Larger k increases the fraction of relabelled transitions and accelerates early learning, but risks overfitting to nearby, easily-achieved goals rather than the true desired goal. $k=4$ is the value recommended in the original paper and is empirically well-balanced: the training curves (Fig. 1) show that the success rate begins rising around 110–120K steps and still on the rise at 250K steps.

5.2 Reward Engineering: A Null Result

Both custom reward functions failed to learn (Table 2), a meaningful null result that reveals two distinct failure modes.

The `dense_energy` result is a textbook case of **reward hacking**. The energy penalty $-\lambda_e \|a_{1:3}\|_2^2$ cannot be recomputed from (g^a, g^d) alone, so during HER relabelling it is silently dropped for relabelled transitions while remaining active for original ones. Facing this inconsistent signal, the agent discovered a simpler optimum: minimise actions entirely, thereby eliminating the energy penalty while accepting the unavoidable sparse failure penalty of -1 . The anomalously low mean energy of 0.03 (vs. 1.20 for sparse+HER) confirms this, and is further corroborated by the simulation video, in which the arm drifts slowly away from both the object and the target, optimising for energy conservation over task completion.

The `approach_progress` reward failed for a different reason. The dense distance terms, gripper-to-object and object-to-goal, distracted the agent from the actual task, trapping it in a local optimum where it remained completely stationary. Staying still minimises movement-related penalties without the risk of incurring additional distance costs, which is a stable but useless fixed point, as confirmed visually by the arm being entirely motionless throughout the episode.

The shared lesson is that HER-compatible reward functions must be recomputable purely from (g^a, g^d) .

5.3 SAC vs. DDPG

SAC+HER achieves 96% while DDPG+HER completely fails (5%). Three algorithmic properties explain this divergence.

First, SAC’s maximum-entropy objective encourages the policy to remain stochastic throughout training, producing diverse arm trajectories that naturally explore the object’s vicinity. DDPG’s deterministic policy relies on additive Gaussian noise for exploration, which is less structured and can degenerate into high-frequency jitter—evidenced by DDPG’s mean energy of 3.00 versus SAC’s 1.20.

Second, SAC’s dual Q-networks reduce value overestimation via a lower-bound target [2]. DDPG’s single Q-network is more susceptible to optimistic bias, potentially producing unstable early-training policy updates before reliable value estimates are established.

Third, the original HER paper notes that DDPG requires three hidden layers and layer normalisation to perform well on Fetch tasks [1]. These architectural modifications were not applied in this experiment, as the goal was to compare the algorithms under identical configurations. This architectural gap likely contributes to DDPG’s failure and represents a straightforward ablation for future work.

5.4 Robustness: Nominal vs. Domain Randomisation

The nominal SAC+HER policy exhibits strong inherent robustness, with success rates of 80–100% across all tested perturbations. Low mass ($\times 0.5$, 80%) is the most damaging condition: a lighter object accelerates more than the policy’s learned force profile expects, causing overshoot or unpredictable sliding. Friction and size variations have minimal impact, suggesting the contact-and-push strategy generalises well across those axes.

The DR-trained policy performs *worse* than the nominal on every tested condition, an unexpected result best explained by **negative transfer**. By training across a wide distribution of physics from the outset, the policy is forced to hedge its behaviour across conditions that may require conflicting strategies, sacrificing peak competence in any single regime. SAC’s maximum-entropy objective already encourages behavioural diversity, meaning the nominal policy acquires implicit robustness without the overhead of distributional training. This finding motivates a curriculum-based approach to domain randomisation rather than uniform sampling from broad bounds from the start (see Section 6).

5.5 Video Analysis

Qualitative review of the simulation recordings provides critical context for the quantitative results and reveals distinct behavioural failure modes across algorithms and reward formulations.

SAC baseline vs. SAC+HER. In `sac_no_her_failure.mp4`, the arm exhibits jittery, upward movements and fails to make any contact with the object, visually confirming the complete absence of a directed learning signal under sparse reward without relabelling. In `sac_her_success.mp4`, the policy smoothly pushes the object to the target. However, once the goal is reached, the end-effector exhibits high-frequency jitter around the object without touching it — indicating the policy has not learned to damp its actuations once the sparse reward condition ($r=0$) is satisfied.

Custom reward failure modes. The videos for both custom reward functions corroborate the reward hacking hypotheses in Section 5.2. In the `dense_energy` variant, the arm drifts slowly away from both the object and the target, confirming that the policy learned to minimise the energy penalty over task completion. In the `approach_progress` variant, the arm remains completely stationary throughout the episode, confirming col-

lapse into a paralysis optimum rather than risking the penalties associated with movement.

DDPG divergence. In `ddpg_her_failure.mp4`, the policy exhibits severe divergence: the arm moves entirely off the table and drops downward, failing to interact with the workspace at all. This catastrophic behaviour aligns with the anomalously high mean energy of 3.00 (Table 3) and demonstrates that DDPG’s deterministic policy, combined with unmitigated Q-value overestimation, produces maximum-bound actions that drive the arm completely out of the useful state space.

6 Proposed Improvement

The robustness experiments reveal a clear motivating finding: the DR-trained policy consistently underperforms the nominal policy due to negative transfer, yet low-mass conditions (80% nominal, 68% DR) remain the hardest perturbation for both. This suggests that broad uniform randomisation from the start of training is counterproductive, but that some form of curriculum over physics difficulty could address both problems.

Automatic Domain Randomisation (ADR) [5] is proposed as a concrete improvement. Rather than uniformly sampling from the full physics range ($\text{mass} \in [0.5, 2.0] \times$, $\text{friction} \in [0.5, 2.0] \times$) from the first episode, ADR begins training at nominal physics and expands the randomisation bounds incrementally only when the agent surpasses a success threshold (e.g., 80%) on the current distribution. Formally, at each boundary-expansion step:

$$\phi_{\min} \leftarrow \phi_{\min} - \delta, \quad \phi_{\max} \leftarrow \phi_{\max} + \delta \quad (5)$$

for each physics parameter ϕ , where δ is a small expansion step.

This curriculum prevents the negative transfer observed in the DR experiments by ensuring the agent first masters the core pushing behaviour before being exposed to edge cases like $0.5 \times$ mass. It directly addresses the failure mode identified in Table 4 and is grounded in the observation that the nominal policy already generalises well to moderate shifts.

Generative AI Disclosure

Generative AI tools were used in the following capacities during this work:

- **Code debugging:** AI assistance was used to identify and resolve bugs in the HER replay buffer implementation, including issues with episode indexing and FIFO overflow handling.

- **Reward engineering:** AI tools were used to assist in debugging the custom reward function implementations in `fetch_push_env.py`.
- **Report writing:** This report was drafted with AI assistance, including structuring sections, refining technical language, and populating tables from experimental results.

All experimental results, implementation decisions, and analysis are the author’s own work. AI-generated content was reviewed and verified throughout.

References

- [1] M. Andrychowicz et al., “Hindsight Experience Replay,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [2] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor,” in *Proc. International Conference on Machine Learning (ICML)*, 2018.
- [3] T. P. Lillicrap et al., “Continuous Control with Deep Reinforcement Learning,” in *Proc. International Conference on Learning Representations (ICLR)*, 2016.
- [4] M. Plappert et al., “Multi-Goal Reinforcement Learning: Challenging Robotics Environments and Request for Research,” *arXiv preprint arXiv:1802.09464*, 2018.
- [5] J. Tobin et al., “Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World,” in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017.
- [6] Farama Foundation, “Gymnasium-Robotics,” 2023. [Online]. Available: <https://robotics.farama.org>
- [7] S. Huang et al., “CleanRL: High-quality Single-file Implementations of Deep Reinforcement Learning Algorithms,” *Journal of Machine Learning Research*, vol. 23, pp. 1–18, 2022.